VOLUME II· MOCK PAPER · WALK-IN SHEET

CMS 710 — The Test Questions, Answered in Full

Four questions set by the lecturer are on file and every one comes from Module 4. This volume answers all four at full exam length, then drills the comparisons that the rest of the course is built from.

Prepared by **Mbosinwa Awunor** · www.mbosinwa.dev

Exam: Wednesday 12 Aug 2026 **Time:** 11:00 – 14:00 **Venue:** the exam hall **Lecturers:** the lecturers

THE FOUR-PART ANSWER SHAPE — USE IT ON EVERY QUESTION

Every question this examiner has set is a **pair of concepts to compare**, and three of the four say "**with an example**". So every answer gets the same four moves, in this order:

1. **Define both terms** — one sentence each, in the module's own words
2. **A comparison table** with a *basis of comparison* column and six or more rows
3. **A code example** showing the difference actually happening, with its output
4. **One closing sentence** on why the difference matters to a language designer

Move 2 carries the most marks per minute. Move 4 is what separates a good answer from a full one — and almost nobody writes it.

1 Question 1 · Control flow defined; sequence vs iteration

(I) CONTROL FLOW, DEFINED

Control flow refers to **the order in which statements, instructions or function calls are executed in a program**. It is governed by **control structures** — the mechanisms that determine the sequence of execution.

Control flow determines **four** things:

1. Which instruction **executes next**
2. Under what conditions execution **changes direction**
3. How **repetition** occurs
4. How **data moves** through program components

It is commonly represented using **flowcharts, control-flow graphs or execution traces**.

Why it matters: without control structures, programs would execute instructions strictly in the order they appear, making it **impossible to implement decision-making, loops or complex algorithms**. The design of control structures significantly influences **program readability, software reliability, maintainability and execution efficiency**.

The four forms of control flow are **sequential execution, selection, iteration and recursion**.

> **Add this and the definition part is complete:** the three principles of control structures — **program behaviour is determined by control structures**; **simplicity improves readability**; **abstraction reduces complexity** (foreach loops, iterators, generators, recursive functions).

(II) SEQUENCE VS ITERATION

Sequential execution is the simplest form of control flow: **statements are executed one after another in the order they appear**. **Iteration** allows **a set of instructions to execute repeatedly until a specified condition is satisfied**; it is commonly called **looping**.

BASIS	SEQUENCE	ITERATION
Definition	Statements executed once each, in written order	A block executed repeatedly until a condition is met
Repetition	None — each statement runs **exactly once**	**Yes** — the same block runs many times
Condition	No condition involved	**Controlled by a condition** that is tested each pass
Direction of flow	Strictly forward, top to bottom	**Returns to the start** of the block
Constructs	Ordinary statements	`for` · `while` · `do–while`
Code length vs work	One statement does one unit of work	**One statement does many units of work**
Termination	Always terminates	**May not terminate** if the condition never becomes false
Purpose	Perform steps in a fixed order	Avoid duplicating code; reduce redundancy

SEQUENCE

```
x = 10
y = 20
z = x + y
print(z)

Execution order:
  1. x = 10
  2. y = 20
  3. z = x + y
  4. print(z)
Four statements,
each run once.
```

ITERATION

```
for i in range(5):
    print(i)


Output:  0 1 2 3 4
One statement, five
executions — the flow
returns to the top of
the block each pass.
```

Closing sentence: sequential execution alone **cannot support decision-making or repetition**, which is precisely why languages provide selection and iteration — iteration **reduces code duplication, improves efficiency and simplifies repetitive tasks**.

2 Question 2 · What is recursion, and how does it differ from iteration?

DEFINITION

Recursion is a **control mechanism in which a function calls itself to solve a problem**. It applies to problems that can be broken into **smaller sub-problems of the same kind**.

A recursive solution has **two essential parts**:

- **Base case** — the condition that **terminates the recursion**
- **Recursive case** — the call to itself **with a smaller problem**

> **Without a base case the recursion never stops**, the call stack grows without bound, and the program fails with a **stack overflow**.

THE EXAMPLE — FACTORIAL

$$n! = n \times (n - 1)!$$

RECURSIVE

```
def factorial(n):
    if n == 1:
        return 1
    return n * \
        factorial(n-1)
```

ITERATIVE

```
def factorial(n):
    result = 1
    for i in range(1, n+1):
        result *= i
    return result
```

TRACE of factorial(5) — recursive

```
factorial(5) = 5 * factorial(4)
factorial(4) = 4 * factorial(3)
factorial(3) = 3 * factorial(2)
factorial(2) = 2 * factorial(1)
factorial(1) = 1              ← BASE CASE reached

unwinding:  2*1=2  3*2=6  4*6=24  5*24 = 120
```

Draw that two-column trace — the winding down to the base case and the unwinding back up. It shows the marker you understand *why* recursion uses more memory: **five calls are alive at once**, each holding its own `n`.

RECURSION VS ITERATION — THE COMPARISON TABLE

BASIS	RECURSION	ITERATION
Mechanism	A function **calls itself** on a smaller sub-problem	A **loop** repeats a block of code
Termination	A **base case**	A **loop condition** that eventually becomes false
Memory use	**Higher** — every call adds a **stack frame**	**Lower** — one set of variables reused
Speed	**Often slower** — function-call overhead	**Faster** — no call overhead
Code size	**Shorter, closer to the mathematical definition**	Longer but more explicit
Readability	Elegant for hierarchical problems, harder for beginners to trace	Straightforward to follow step by step
Failure mode	**Stack overflow** if the base case is missing or never reached	**Infinite loop** if the condition never fails
Best suited to	**Trees, graphs**, divide-and-conquer, hierarchical structures	Simple counted or condition-controlled repetition
State	Held **implicitly on the call stack**	Held **explicitly in loop variables**

ADVANTAGES AND DISADVANTAGES, AS THE MODULE LISTS THEM

ADVANTAGES OF RECURSION	DISADVANTAGES
Elegant solutions	Higher memory consumption
Natural representation of hierarchical structures	Risk of stack overflow
Useful for trees and graphs	Often slower than iteration

> **The closing sentence.** Recursion and iteration are **computationally equivalent** — anything one can do, the other can too. The choice is therefore not about power but about **clarity versus cost**: recursion buys a solution that mirrors the structure of the problem, and pays for it in **stack memory and call overhead**. Which is why a language designer offering **excellent recursion support**, as Python, JavaScript and C++ all do, must also give programmers loops.

3 Question 3 · Global vs local scope, and variable lifetime

DEFINITIONS FIRST

Scope determines **where a variable can be accessed within a program**. It is a fundamental concept because it **controls variable visibility and lifetime**.

Global scope — the variable is **accessible throughout the program**, from any function or block.
Local scope — the variable is **accessible only within the function or block in which it is declared**.

The module names a third: **block scope**, where variables exist only inside a specific block — in JavaScript, `let age = 20;` declared inside an `if` block **produces an error** if read outside it.

BASIS	GLOBAL SCOPE	LOCAL SCOPE
Where declared	**Outside** all functions and blocks	**Inside** a function or block
Visibility	Everywhere in the program	Only within its own function or block
Created when	The **program starts**	The **function is called**
Destroyed when	The **program ends**	The **function returns**
Lifetime	**Entire program execution**	**Only while the function executes**
Memory area	Static / global storage	The **call stack**
Name conflicts	Risk of clashes across the program	Same name may be reused safely in different functions
Effect on modularity	**Weakens** it — any function may change the value	**Supports** it — data stays where it is used

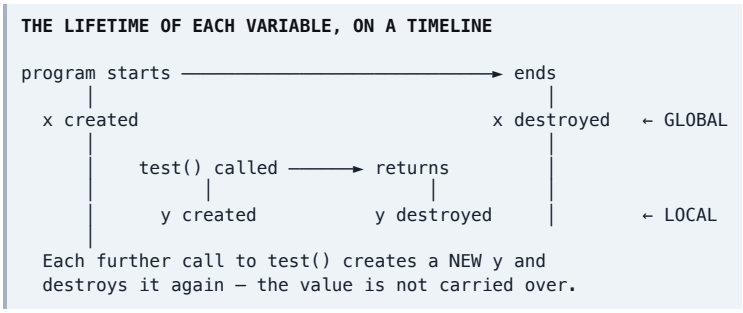
THE EXAMPLE

```
x = 10                  # GLOBAL — visible everywhere

def test():
    y = 20              # LOCAL — visible only in test()
    print(x)            # legal: globals are visible here
    print(y)            # legal

test()
print(x)                # legal — 10
print(y)                # ERROR: y does not exist here
```

USING SCOPE TO EXPLAIN LIFETIME — WHAT THE QUESTION ACTUALLY WANTS

Variable lifetime is **the period during which a variable exists in memory**. It depends on **scope, the storage allocation method and the program execution state**.

> **The link, stated directly. Scope and lifetime are two views of the same declaration.** Scope is a **spatial** property — *where in the program text* the name is visible. Lifetime is a **temporal** property — *for how long during execution* the storage exists. They are connected because **the scope in which a variable is declared determines the storage it is given, and that storage determines how long it lives**:
>
> - A variable declared in **global scope** is allocated in **static storage when the program loads**, so its lifetime is the **entire execution**.
> - A variable declared in **local scope** is allocated on the **call stack when its function is entered** and **reclaimed when the function returns**, so its lifetime is **just that one call**.



The consequence worth stating: because a local variable is destroyed on return, it **cannot remember anything between calls**; because a global survives, **any function can change it**, which is why globals are convenient but harmful to modularity.

IMPORTANCE OF SCOPE — THE FOUR POINTS

1. **Prevents naming conflicts** — two functions may each use `i` without interference
2. **Enhances security** — data is not exposed beyond where it is needed
3. **Improves maintainability** — a change to a local variable cannot affect distant code
4. **Supports modular design** — modules communicate through parameters, not shared state

4 Question 4 · Pass by value vs pass by reference

DEFINITIONS

Functions often require data from other parts of a program, and languages provide **parameter-passing mechanisms** for supplying it.

Pass-by-value: a **copy of the argument is passed** to the function. **Changes made inside the function do not affect the original variable.**

Pass-by-reference: the function receives a **reference to the original variable**. **Changes made inside the function affect the original.**

BASIS	PASS BY VALUE	PASS BY REFERENCE
What is passed	A **copy** of the value	A **reference to** the original variable
C++ syntax	`void f(int x)`	`void f(int &x)`
Effect on original	**Unchanged**	**Modified**
Memory	Extra memory for the copy	No copy — more memory-efficient for large data
Speed	Slower for large objects (copying cost)	Faster — nothing is duplicated
Safety	**Safer** — the caller's data cannot be corrupted	**Riskier** — unintended changes are possible
Side effects	None	**Yes** — this is its purpose
Use when	The function only needs to **read** the value	The function must **modify** the caller's variable, or the data is large

THE EXAMPLE, WITH OUTPUT

```
PASS BY VALUE
#include <iostream>
using namespace std;

void increment(int x) {      // a COPY
    x++;
    cout << "inside:  " << x << endl;
}

int main() {
    int num = 10;
    increment(num);
    cout << "outside: " << num << endl;
    return 0;
}

OUTPUT   inside:  11
         outside: 10      ← original UNCHANGED
```

```
PASS BY REFERENCE
#include <iostream>
using namespace std;

void increment(int &x) {     // the ACTUAL variable
    x++;
    cout << "inside:  " << x << endl;
}

int main() {
    int num = 10;
    increment(num);
    cout << "outside: " << num << endl;
    return 0;
}

OUTPUT   inside:  11
         outside: 11      ← original MODIFIED
```

The two programs differ by a single character — the `&` — and that is exactly what makes them worth putting side by side in an answer.

PYTHON'S MECHANISM — THE EXTRA MARK

Python uses an approach usually described as **pass-by-object-reference**: **objects are passed by reference to object values**.

```
def modify(lst):
    lst.append(100)         # the ORIGINAL list is modified

numbers = [1, 2, 3]
modify(numbers)
print(numbers)              # [1, 2, 3, 100]

def reassign(lst):
    lst = [9, 9, 9]         # rebinds the local name only

reassign(numbers)
print(numbers)              # [1, 2, 3, 100] — unchanged
```

The subtlety worth one sentence: in Python, **mutating** an object through the parameter affects the caller, but **rebinding** the parameter to a new object does not — which is why it is neither pure pass-by-value nor pure pass-by-reference.

The closing sentence. The choice of parameter-passing mechanism is a **language design decision balancing safety against efficiency and expressive power**: pass-by-value protects the caller's data but copies it, pass-by-reference avoids the copy and permits a function to return results through its parameters, at the cost of **side effects that make programs harder to reason about**. This is why the module rates **parameter-passing complexity as low in Python, moderate in JavaScript and high in C++** — C++ gives the programmer every option and the responsibility that comes with it.

5 The other comparisons this course is built from

Every question set so far has been a **comparison**. These are the remaining pairs the four modules set up — learn the **basis words** in the left column and the tables write themselves.

SYNTAX VS SEMANTICS

Concern	Structure — how it is written	Meaning — what it does
Checked by	The parser	Semantic analysis / runtime
Example error	`= x 10`	Dividing by a variable that is zero

STATIC VS DYNAMIC TYPING

Checked	At **compile** time	At **run** time
Languages	C++	Python, JavaScript
Gains	Early errors, reliability, speed	Flexibility, faster development
Costs	Verbosity, less flexibility	Runtime errors, less type safety

STRONG VS WEAK TYPING

Conversions	Prevented	Allowed implicitly
Example	Python `age + "years"` → error	JS `"5" + 2` → `"52"`

PRIMITIVE VS COMPOSITE VS USER-DEFINED TYPES

Primitive	Provided by the language, **cannot be decomposed** — int, float, char, bool
Composite	Combine primitives into one structure — arrays, lists, tuples, dictionaries, objects
User-defined	Created by the programmer — structs, classes, enumerations, records

ARRAY VS LIST

Size	Fixed	**Dynamic** — grows and shrinks
Memory	Contiguous	Extra overhead
Access	**Fast**, by index	Slower for some operations
Insert/delete	**Costly**	Flexible

STACK VS QUEUE

Rule	LIFO	FIFO
Operations	Push · Pop · Peek · IsEmpty	Enqueue · Dequeue · Front · IsEmpty
Uses	Browser history, undo, function calls, expression evaluation	Bank queues, print queues, OS scheduling, network packets

PARSE TREE VS ABSTRACT SYNTAX TREE

Represents	The **full grammatical structure** per the grammar	A **simplified** structure preserving meaning
Produced by	**Syntax analysis** (parsing)	**Semantic analysis**
Detail	Keeps every grammar symbol	**Removes unnecessary grammar details**
Used by	The parser	Compilers, interpreters, static analysis, optimizers, AI code assistants

IMPERATIVE VS FUNCTIONAL CONTROL FLOW

Describes	**How** a task is performed	**What** is to be computed
Example	`total = 0; for n in numbers: total += n`	`total = sum(numbers)`
Gains	Explicit control	Simpler code, abstraction, **fewer side effects**

COMPILED VS INTERPRETED

Translation	**Whole program before** execution	**Statement by statement during** execution
Speed	**Faster** at run time	Slower
Errors	Found before running	Found when the line is reached
Example	C++	Python · JavaScript (JIT)

ADT VS DATA STRUCTURE

Specifies	**What** — the data and permitted operations	**How** — the actual organization in memory
Example	Stack ADT: Push, Pop, Peek, IsEmpty	That stack built on an array or a linked list

6 Mock paper — sit this closed-book

RIVERS STATE UNIVERSITY · PGD COMPUTER SCIENCE · CMS 710 — PRINCIPLES OF PROGRAMMING LANGUAGES
Mock examination · Time allowed: 3 hours · **Answer any FIVE questions** · Each question carries 20 marks · Illustrate your answers with examples in Python, JavaScript or C++

Q	QUESTION	MARKS
1	**(a)** Define a programming language and state the three perspectives from which it may be viewed. **(b)** Differentiate between **syntax** and **semantics** with examples. **(c)** State and explain the **five goals** of programming language design.	5 + 6 + 9
2	**(a)** Trace the **evolution** of programming languages through the four generations, giving an example and two characteristics of each. **(b)** What is a **language specification** and what does it define? **(c)** Write the BNF rule for an assignment statement and explain its meaning.	10 + 5 + 5
3	**(a)** Define a **data type** and state the four things it specifies. **(b)** Differentiate between **primitive, composite and user-defined** data types with examples in three languages. **(c)** Compare **static and dynamic typing**, and separately **strong and weak typing**, in tables.	5 + 7 + 8
4	**(a)** What is **data abstraction**? Illustrate with an analogy and state four benefits. **(b)** Define an **Abstract Data Type** and illustrate with the Stack ADT. **(c)** Differentiate an ADT from a data structure.	7 + 8 + 5

Q	QUESTION	MARKS
5	**(a)** Differentiate between a **stack** and a **queue**, giving operations and two real-life uses of each. **(b)** With diagrams, distinguish a **parse tree** from an **abstract syntax tree**. **(c)** Explain the role of trees in the four phases of language processing.	7 + 7 + 6
6	**(a)** Define **control flow** and state what it determines. **(b)** Differentiate between **sequence and iteration** with examples. **(c)** What is **recursion**, and how does it differ from iteration? Use factorial to illustrate.	5 + 7 + 8
7	**(a)** Differentiate between **global and local scope**, and use that distinction to explain **variable lifetime**. **(b)** Compare **pass by value** and **pass by reference** with a C++ example of each. **(c)** Explain Python's **pass-by-object-reference**.	8 + 8 + 4

Where each answer lives: Q1, Q2 → Vol I §2 · Q3 → Vol I §3 · Q4 → Vol I §3–§4 · Q5 → Vol I §4 · **Q6, Q7 → this volume §1–§4** — those two are the real test questions, reassembled.

7 Rapid-fire recall — cover the right column

PROMPT	ANSWER
Programming language, in one line	A formal language of symbols, keywords, syntax rules and semantic rules used to instruct a computer
Three perspectives on a language	Communication · problem-solving · formal system
The four generations	1GL machine · 2GL assembly · 3GL high-level · 4GL logic/AI
Five design goals	Readability · writability · reliability · maintainability · efficiency
Syntax vs semantics	Structure vs meaning
What a specification defines	Syntax, semantics, data types, operators, control structures, libraries, runtime behaviour
BNF assignment rule	`<assignment> ::= <identifier> = <expression>`
Three principles of language definition	Formal rules govern every language · simplicity vs expressiveness · efficiency vs safety
A data type specifies…	Values · operations · memory required · interpretation
Three type categories	Primitive · composite · user-defined
Static vs dynamic typing	Checked at compile time vs at run time
Strong vs weak typing	Prevents vs allows implicit conversions
Where Python sits on both axes	**Dynamic and strong**
Data abstraction	Hiding implementation, exposing essentials — the car analogy
Four benefits of abstraction	Simplicity · reusability · maintainability · security

PROMPT	ANSWER
Data structure, in one line	A method of organizing and storing data so it can be accessed and modified efficiently
ADT, in one line	A logical description of data and operations **without** implementation details
Four ADT advantages	Abstraction · reusability · maintainability · modularity
Stack ADT operations	Push · Pop · Peek · IsEmpty
Queue ADT operations	Enqueue · Dequeue · Front · IsEmpty
Three data-structure principles	Efficient organization · abstraction of complexity · time–space trade-off
Parse tree vs AST	Full grammatical structure vs simplified, meaning-preserving
The four language-processing phases	Lexical analysis → tokens · parsing → parse trees · semantic analysis → ASTs · code generation
Control flow, in one line	The order in which statements, instructions or function calls are executed
Four forms of control flow	Sequential · selection · iteration · recursion
Three types of selection	Single · double · multiple
The three loops	`for` known count · `while` condition · `do–while` at least once
Two parts of a recursive solution	Base case · recursive case
Three types of scope	Global · local · block
Variable lifetime depends on…	Scope · storage allocation method · program execution state
Three parameter-passing mechanisms	By value · by reference · Python's by object reference
The IPO model	Input → Processing → Output
Structured programming emphasizes…	Sequence, selection, iteration — avoiding uncontrolled GOTO

8 Walk-in sheet — the last ten minutes

THE FOUR TEST QUESTIONS, IN ONE LINE EACH

1. **Control flow** = the order in which statements, instructions or function calls are executed. **Sequence** runs each statement once in order; **iteration** runs a block repeatedly under a condition.
2. **Recursion** = a function calling itself, with a **base case** and a **recursive case**. Versus iteration: **more memory (stack frames), slower, shorter code, risks stack overflow** — best for trees and hierarchies.
3. **Global** = visible everywhere, lives for the whole program; **local** = visible in one function, lives only while it runs. **Scope is spatial, lifetime is temporal, and scope determines lifetime.**
4. **By value** = a copy, original unchanged; **by reference** (`&`) = the actual variable, original modified. Python is **pass-by-object-reference**.

THE FOUR-MOVE ANSWER SHAPE

Define both → table with a basis column → code example with output → one sentence on why it matters to a language designer.

THE THREE TABLES TO HAVE READY

- **Python / JavaScript / C++:** block structure = indentation / braces / braces · typing = dynamic / dynamic / **static** · compilation = interpreted / JIT / **compiled** · readability = very high / high / moderate · speed = moderate / moderate / **high**
- **Type axes:** Python dynamic+strong · JavaScript dynamic+weak · C++ static+strong
- **Structures:** stack LIFO, queue FIFO, array fixed, list dynamic

THREE MOVES IN THE FIRST FIVE MINUTES

1. Read all the questions and pick the five with the most **comparisons** — those are the fastest to answer well.
2. For each chosen question, sketch the **table headings** in the margin before writing prose.
3. Name **all three languages** wherever a comparison is invited; the whole course is built on that trio.

IF YOUR MIND GOES BLANK

Write the **definition of each term**, then draw a **two-column table** and fill the basis column with: *definition* · *mechanism* · *memory* · *speed* · *safety* · *use when*. Those six words fit almost every pair on this syllabus.